

Project 11: AES Implementation

A small Java program for AES encryption and decryption

Student Name: Georgios Panormitis Latos
Course: DA563A Introduction to Computer Security
Teacher: Qinghua Wang
Date: 28/03/2026

1 Overview

This report presents a small Java program that performs AES encryption and decryption. The project was made for Project 11 in the course material, where the main requirement was to encrypt the message **Introduction to Computer Security** and then decrypt it again with the same key.

I used Java because it already provides the Java Cryptography Architecture, which makes it possible to build a working encryption program without implementing the AES algorithm from the lowest level. I also added a small Swing interface so the program is easier to use and test.

2 Introduction

In this project I implemented a small Java program that can perform AES encryption and decryption. AES stands for Advanced Encryption Standard and it is one of the most common symmetric encryption algorithms used today. In a symmetric cryptosystem, the same secret key is used both when data is encrypted and when it is decrypted again.

The assignment for Project 11 was to write a program that can encrypt and decrypt a message with AES. The instructions also suggested that Java and the Java Cryptography Architecture could be used. For the demonstration part, the message **Introduction to Computer Security** had to be encrypted and then decrypted again with a key of my own choice.

I selected Java because it already contains ready classes for cryptographic work, which made it possible to focus on how the program behaves instead of implementing the AES algorithm from the lowest level. I also added a simple graphical interface with text areas and buttons so that the program is easy to test.

3 Implementation

3.1 Program Structure

The implementation is divided into a small number of Java files. The class **AesCryptoService** is responsible for the actual cryptographic operations. It generates a 128 bit AES key with **KeyGenerator** and stores that key for the current program session. The program uses the transformation **AES/ECB/PKCS5Padding**. The same key is then reused for both encryption and decryption.

3.2 Encryption And Decryption

When the user presses the Encrypt button, the program reads the plaintext from the input area, converts it to UTF 8 bytes, and encrypts it. The encrypted bytes are then converted into Base64 text. I decided to show the result in Base64 instead of raw binary because it is much more readable in the user interface. When the Decrypt button is pressed, the program reads the Base64 ciphertext, decodes it back into bytes, and decrypts it with the same AES key.

3.3 Interface

The GUI was written with Swing because it is included in Java and does not need extra libraries. The interface contains one input text box, one output text box, one key field, and two buttons. There is also a small status line that tells the user whether the last action was successful or whether there was an error, for example if invalid ciphertext is entered for decryption.

For this assignment I used ECB mode because it matches the recommendation in the instructions. In real world applications, ECB mode is usually not the best choice for larger or repeated data patterns, but for a short classroom demonstration with one message it is acceptable and it keeps the example easy to understand.

4 Program Design and GUI

The program starts with the required sentence already placed in the input area, which makes the demonstration direct and simple. A new AES key is generated at startup, and the Base64 form of that key is displayed in the window so it is clear which key is being used during the session.

The workflow is very basic. First, the user presses Encrypt. The program then shows the encrypted message in the output area. After that, the user can press Decrypt, and the original plaintext appears again. This makes it easy to confirm that the round trip works correctly.

I also added a small command line check class called `AesRoundTripCheck`. This file is useful because it verifies the same encryption and decryption logic without opening the GUI. I also added a second test file so that I could check how the program behaves with empty text, multiline text, punctuation, and UTF 8 text before submission.

5 Results and Discussion

The program worked as expected during testing. When the default message was encrypted, the program produced a Base64 ciphertext string in the output field. When that ciphertext was decrypted, the original text `Introduction to Computer Security` was recovered correctly.

I also tested the program by compiling the Java files and running the command line verification class. The round trip check printed the original text, the encrypted text, the decrypted text, and finally a boolean result showing that the decrypted text matched the original exactly. I then added a small self test file that checks several normal and edge cases, such as the required sentence, an empty string, a different input text, a Unicode string, and invalid ciphertext input. This gave me more confidence that the program logic was stable.

The graphical interface also behaved correctly in the normal use case. The key was shown clearly, the Encrypt button generated ciphertext, and the Decrypt button recovered the plaintext again. If the user tried to decrypt invalid text, the program showed a small error message instead of crashing. That makes the program more robust even though it is still a simple assignment program.

The main results can be summarized in a simple way:

- The required assignment text was encrypted successfully.
- The ciphertext could be decrypted back to the original message.
- The GUI worked correctly for normal user input.
- The program also handled empty text, multiline text, punctuation, and UTF 8 text in the self test.
- Invalid ciphertext input was rejected instead of causing the program to crash.

6 Conclusion

To conclude, this project shows a working AES implementation in Java using the Java Cryptography Architecture. The program can encrypt and decrypt text successfully with the same secret key, and it demonstrates the required sentence from the assignment without difficulty.

The implementation was not very large, but it still includes the main parts needed for a useful demonstration: key generation, encryption, decryption, a simple interface, and verification. I think this project was a good practical exercise because it connected the

theory of symmetric cryptography with an actual working program.

Appendix: Annotated Source Code

The following source files are included as a part of the submission. I added comments by keeping the code structure simple and by choosing descriptive names so that the program is easy to follow.

AesCryptoService.java

```
import java.nio.charset.StandardCharsets;
import java.security.GeneralSecurityException;
import java.util.Base64;

import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;

public class AesCryptoService {
    private static final String ALGORITHM = "AES";
    private static final String TRANSFORMATION = "AES/ECB/PKCS5Padding";

    private SecretKey activeKey;

    public AesCryptoService() throws GeneralSecurityException {
        generateKey();
    }

    public SecretKey generateKey() throws GeneralSecurityException {
        KeyGenerator keyGenerator = KeyGenerator.getInstance(ALGORITHM);
        keyGenerator.init(128);
        activeKey = keyGenerator.generateKey();
        return activeKey;
    }

    public String encrypt(String plaintext) throws GeneralSecurityException {
        if (plaintext == null) {
            throw new IllegalArgumentException("Plaintext cannot be null.");
        }

        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        cipher.init(Cipher.ENCRYPT_MODE, activeKey);
        byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(encryptedBytes);
    }

    public String decrypt(String base64Ciphertext) throws GeneralSecurityException {
        if (base64Ciphertext == null) {
            throw new IllegalArgumentException("Ciphertext cannot be null.");
        }

        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        cipher.init(Cipher.DECRYPT_MODE, activeKey);
        byte[] ciphertextBytes = Base64.getDecoder().decode(base64Ciphertext);
```

```

        byte[] decryptedBytes = cipher.doFinal(ciphertextBytes);
        return new String(decryptedBytes, StandardCharsets.UTF_8);
    }

    public String getKeyBase64() {
        return Base64.getEncoder().encodeToString(activeKey.getEncoded());
    }
}

```

AesGuiApp.java

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.Graphics2D;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.RenderingHints;
import java.awt.image.BufferedImage;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.GeneralSecurityException;

import javax.imageio.ImageIO;
import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JComponent;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import javax.swing.WindowConstants;

public class AesGuiApp {
    private static final String DEFAULT_MESSAGE = "Introduction to Computer Security";
    private static final Dimension CAPTURE_SIZE = new Dimension(920, 700);
    private static final Color BACKGROUND_COLOR = new Color(245, 247, 250);
    private static final Color PANEL_COLOR = Color.WHITE;
    private static final Color SUCCESS_COLOR = new Color(24, 110, 57);
    private static final Color ERROR_COLOR = new Color(176, 39, 39);
    private static final Color TEXT_COLOR = new Color(28, 37, 48);

    private final AesCryptoService cryptoService;
    private final JTextArea inputArea;
    private final JTextArea outputArea;
    private final JTextField keyField;
    private final JLabel statusLabel;

    public AesGuiApp() throws GeneralSecurityException {

```

```

        cryptoService = new AesCryptoService();
        inputArea = createTextArea(true);
        outputArea = createTextArea(false);
        keyField = new JTextField(cryptoService.getKeyBase64());
        statusLabel = new JLabel("Ready. Press Encrypt to encrypt the text.");

        inputArea.setText(DEFAULT_MESSAGE);
        outputArea.setText("");
        keyField.setEditable(false);
        keyField.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));
        keyField.setBackground(new Color(248, 250, 252));
        keyField.setForeground(TEXT_COLOR);
        keyField.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(208, 214, 222)),
            BorderFactory.createEmptyBorder(8, 10, 8, 10)));

        statusLabel.setForeground(new Color(82, 89, 98));
        statusLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 13));
    }

    public static void main(String[] args) throws Exception {
        if (args.length >= 2 && "--capture".equals(args[0])) {
            Path outputDirectory = Path.of(args[1]);
            SwingUtilities.invokeLater(() -> {
                try {
                    captureScreenshots(outputDirectory);
                } catch (Exception exception) {
                    throw new RuntimeException(exception);
                }
            });
            return;
        }

        SwingUtilities.invokeLater(() -> {
            try {
                AesGuiApp app = new AesGuiApp();
                app.show();
            } catch (GeneralSecurityException exception) {
                throw new RuntimeException(exception);
            }
        });
    }

    private static void captureScreenshots(Path outputDirectory) throws Exception {
        Files.createDirectories(outputDirectory);

        // Each screenshot uses a fresh app instance so the state is predictable.
        AesGuiApp initialApp = new AesGuiApp();
        saveComponentImage(initialApp.createRootPanel(), outputDirectory.resolve("gui-launch.png"));

        AesGuiApp encryptedApp = new AesGuiApp();
        encryptedApp.encryptCurrentInput();
        saveComponentImage(encryptedApp.createRootPanel(), outputDirectory.resolve("gui-encrypted.png"));

        AesGuiApp decryptedApp = new AesGuiApp();
        decryptedApp.encryptCurrentInput();
        decryptedApp.decryptCurrentOutput();
    }

```

```

        saveComponentImage(decryptedApp.createRootPanel(), outputDirectory.resolve("gui-
            decrypted.png"));
    }

    private static void saveComponentImage(JComponent component, Path outputPath) throws
        IOException {
        component.setSize(CAPTURE_SIZE);
        component.doLayout();
        layoutRecursively(component);

        // Render the Swing component directly into an image file for the report.
        BufferedImage image = new BufferedImage(
            CAPTURE_SIZE.width,
            CAPTURE_SIZE.height,
            BufferedImage.TYPE_INT_RGB);
        Graphics2D graphics = image.createGraphics();
        graphics.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.
            VALUE_ANTIALIAS_ON);
        graphics.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.
            VALUE_TEXT_ANTIALIAS_ON);
        component.paint(graphics);
        graphics.dispose();

        ImageIO.write(image, "png", outputPath.toFile());
    }

    private static void layoutRecursively(JComponent component) {
        for (java.awt.Component child : component.getComponents()) {
            child.doLayout();
            if (child instanceof JComponent childComponent) {
                layoutRecursively(childComponent);
            }
        }
    }

    private JTextArea createTextArea(boolean editable) {
        JTextArea area = new JTextArea(6, 34);
        area.setEditable(editable);
        area.setLineWrap(true);
        area.setWrapStyleWord(true);
        area.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 15));
        area.setForeground(TEXT_COLOR);
        area.setBackground(new Color(248, 250, 252));
        area.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        return area;
    }

    private void show() {
        JFrame frame = new JFrame("AES Encryption and Decryption");
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        frame.setContentPane(createRootPanel());
        frame.pack();
        frame.setMinimumSize(new Dimension(860, 660));
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }

    private JComponent createRootPanel() {
        JPanel rootPanel = new JPanel(new BorderLayout(0, 18));
    }

```



```

        rootPanel.setPreferredSize(CAPTURE_SIZE);
        rootPanel.setBorder(BorderFactory.createEmptyBorder(22, 22, 22, 22));
        rootPanel.setBackground(BACKGROUND_COLOR);
        rootPanel.add(createHeaderPanel(), BorderLayout.NORTH);
        rootPanel.add(createContentPanel(), BorderLayout.CENTER);
        return rootPanel;
    }

    private JComponent createHeaderPanel() {
        JPanel headerPanel = new JPanel();
        headerPanel.setBackground(BACKGROUND_COLOR);
        headerPanel.setLayout(new BoxLayout(headerPanel, BoxLayout.Y_AXIS));

        JLabel titleLabel = new JLabel("AES Implementation");
        titleLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 24));
        titleLabel.setForeground(TEXT_COLOR);

        JLabel subtitleLabel = new JLabel("Encrypt and decrypt text using Java
            Cryptography Architecture.");
        subtitleLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 14));
        subtitleLabel.setForeground(new Color(88, 96, 105));

        headerPanel.add(titleLabel);
        headerPanel.add(Box.createVerticalStrut(6));
        headerPanel.add(subtitleLabel);
        return headerPanel;
    }

    private JComponent createContentPanel() {
        JPanel outerPanel = new JPanel(new BorderLayout());
        outerPanel.setBackground(PANEL_COLOR);
        outerPanel.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(215, 220, 226)),
            BorderFactory.createEmptyBorder(18, 18, 18, 18)));

        JPanel contentPanel = new JPanel(new GridBagLayout());
        contentPanel.setBackground(PANEL_COLOR);

        GridBagConstraints constraints = new GridBagConstraints();
        constraints.gridx = 0;
        constraints.gridy = 0;
        constraints.weightx = 1.0;
        constraints.fill = GridBagConstraints.HORIZONTAL;
        constraints.insets = new Insets(0, 0, 8, 0);
        contentPanel.add(createSectionLabel("Generated AES Key (Base64)", constraints);

        constraints.gridy++;
        constraints.insets = new Insets(0, 0, 18, 0);
        contentPanel.add(keyField, constraints);

        constraints.gridy++;
        constraints.insets = new Insets(0, 0, 8, 0);
        contentPanel.add(createSectionLabel("Enter Text", constraints);

        constraints.gridy++;
        constraints.fill = GridBagConstraints.BOTH;
        constraints.weighty = 0.45;
        constraints.insets = new Insets(0, 0, 18, 0);
        contentPanel.add(wrapInScrollPane(inputArea), constraints);
    }

```

```

        constraints.gridy++;
        constraints.weighty = 0.0;
        constraints.fill = GridBagConstraints.HORIZONTAL;
        constraints.insets = new Insets(0, 0, 8, 0);
        contentPanel.add(createSectionLabel("Encrypted / Decrypted Text"), constraints);

        constraints.gridy++;
        constraints.fill = GridBagConstraints.BOTH;
        constraints.weighty = 0.45;
        constraints.insets = new Insets(0, 0, 18, 0);
        contentPanel.add(wrapInScrollPane(outputArea), constraints);

        constraints.gridy++;
        constraints.weighty = 0.0;
        constraints.fill = GridBagConstraints.HORIZONTAL;
        constraints.insets = new Insets(0, 0, 14, 0);
        contentPanel.add(createButtonPanel(), constraints);

        constraints.gridy++;
        constraints.insets = new Insets(0, 0, 0, 0);
        contentPanel.add(statusLabel, constraints);

        outerPanel.add(contentPanel, BorderLayout.CENTER);
        return outerPanel;
    }

    private JLabel createSectionLabel(String text) {
        JLabel label = new JLabel(text);
        label.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 15));
        label.setForeground(TEXT_COLOR);
        return label;
    }

    private JScrollPane wrapInScrollPane(JTextArea textArea) {
        JScrollPane scrollPane = new JScrollPane(textArea);
        scrollPane.setBorder(BorderFactory.createLineBorder(new Color(208, 214, 222)));
        scrollPane.setBackground(new Color(248, 250, 252));
        return scrollPane;
    }

    private JComponent createButtonPanel() {
        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(PANEL_COLOR);
        buttonPanel.setLayout(new BoxLayout(buttonPanel, BoxLayout.X_AXIS));

        JButton encryptButton = new JButton("Encrypt");
        encryptButton.addActionListener(event -> encryptCurrentInput());

        JButton decryptButton = new JButton("Decrypt");
        decryptButton.addActionListener(event -> decryptCurrentOutput());

        buttonPanel.add(encryptButton);
        buttonPanel.add(Box.createHorizontalStrut(10));
        buttonPanel.add(decryptButton);
        buttonPanel.add(Box.createHorizontalGlue());
        return buttonPanel;
    }
}

```

```

private void encryptCurrentInput() {
    try {
        String plaintext = inputArea.getText();
        if (plaintext.isBlank()) {
            showError("Please enter text before pressing Encrypt.");
            return;
        }

        String encryptedText = cryptoService.encrypt(plaintext);
        outputArea.setText(encryptedText);
        showSuccess("Encryption completed successfully.");
    } catch (GeneralSecurityException exception) {
        showError("Encryption failed: " + exception.getMessage());
    }
}

private void decryptCurrentOutput() {
    try {
        String ciphertext = outputArea.getText().trim();
        if (ciphertext.isEmpty()) {
            showError("Please encrypt text first or paste Base64 ciphertext in the
                output box.");
            return;
        }

        String decryptedText = cryptoService.decrypt(ciphertext);
        outputArea.setText(decryptedText);
        showSuccess("Decryption completed successfully.");
    } catch (Exception exception) {
        showError("Decryption failed. Make sure the text is valid Base64 ciphertext
            .");
    }
}

private void showSuccess(String message) {
    statusLabel.setText(message);
    statusLabel.setForeground(SUCCESS_COLOR);
}

private void showError(String message) {
    statusLabel.setText(message);
    statusLabel.setForeground(ERROR_COLOR);
}
}

```

AesRoundTripCheck.java

```

public class AesRoundTripCheck {
    public static void main(String[] args) throws Exception {
        String originalText = "Introduction to Computer Security";

        AesCryptoService cryptoService = new AesCryptoService();
        String encryptedText = cryptoService.encrypt(originalText);
        String decryptedText = cryptoService.decrypt(encryptedText);

        System.out.println("AES key (Base64): " + cryptoService.getKeyBase64());
        System.out.println("Original text: " + originalText);
        System.out.println("Encrypted text (Base64): " + encryptedText);
    }
}

```

```

        System.out.println("Decrypted text: " + decryptedText);
        System.out.println("Round-trip match: " + originalText.equals(decryptedText));
    }
}

```

AesServiceSelfTest.java

```

import java.util.Base64;

public class AesServiceSelfTest {
    public static void main(String[] args) throws Exception {
        AesCryptoService cryptoService = new AesCryptoService();

        check(Base64.getDecoder().decode(cryptoService.getKeyBase64()).length == 16,
            "Generated key is 128 bits.");

        assertRoundTrip(cryptoService, "Introduction to Computer Security",
            "Required assignment message round trips correctly.");
        assertRoundTrip(cryptoService, "",
            "Empty string round trips correctly.");
        assertRoundTrip(cryptoService, "    leading and trailing spaces    ",
            "Text with surrounding spaces round trips correctly.");
        assertRoundTrip(cryptoService, "AES test message 123",
            "Different ASCII input round trips correctly.");
        assertRoundTrip(cryptoService, "Kalimera cryptography",
            "Plain text with different content round trips correctly.");
        assertRoundTrip(cryptoService, "Line 1\nLine 2\nLine 3",
            "Multiline text round trips correctly.");
        assertRoundTrip(cryptoService, "Tabs\tspaces\tand punctuation !?.,:;()[]{}\"",
            "Text with mixed spacing and punctuation round trips correctly.");
        assertRoundTrip(cryptoService, "UTF 8 test: caf\u00e9, na\u00efve, jalape\u00f1o",
            "UTF 8 text round trips correctly.");

        boolean invalidCiphertextRejected = false;
        try {
            cryptoService.decrypt("this-is-not-valid-base64");
        } catch (Exception exception) {
            invalidCiphertextRejected = true;
        }
        check(invalidCiphertextRejected, "Invalid ciphertext is rejected.");

        System.out.println("All AES self tests passed.");
    }

    private static void assertRoundTrip(AesCryptoService cryptoService, String plainText,
        String message)
        throws Exception {
        String encryptedText = cryptoService.encrypt(plainText);
        String decryptedText = cryptoService.decrypt(encryptedText);

        check(!encryptedText.equals(plainText), "Ciphertext differs from plaintext for: "
            + message);
        check(plainText.equals(decryptedText), message);
    }

    private static void check(boolean condition, String message) {
        if (!condition) {

```

```
        throw new IllegalStateException("Test failed: " + message);
    }
    System.out.println("PASS: " + message);
}
}
```